USO INTERNO — LABORATORIO PERSONALE

exa-dune

Framework Multi-Protocollo per Penetration Testing

Versione 3.0 — 22 Marzo 2026

Manuale operativo completo per l'utilizzo di exa-dune in ambienti IoT e VoIP. Questa versione documenta tutti i 51 subcomandi, i 14 script ausiliari Python/JS/Shell, il sistema di propagazione credenziali, i profili auto dedicati (voip-gateway, ip-camera, iot-adapter), il database SQLite, il scheduling cron, l'analisi firmware e molto altro.

51	14	11.7k	6
SUBCOMANDI	SCRIPT AUSILIARI	RIGHE CODICE	VENDOR CVE

Autore: Security Lab — Piattaforma: Kali Linux 6.18 — GitHub: github.com/Sal1699/exa-dune

⚠ CONFIDENZIALE — USO ESCLUSIVAMENTE IN AMBIENTI AUTORIZZATI ⚠

Indice dei Contenuti

1. Introduzione e Concetti Base 3

2. Installazione e Configurazione 5

3. Architettura e Struttura 7

4. Flag Globali e Opzioni 9

5. Profili Auto (voip-gateway, ip-camera, iot-adapter) 11

6. Ricognizione e Scansione 13

- 6.1 scan 13
- 6.2 network-sweep ✦ NUOVO 15
- 6.3 web, ssl, dns, js, http 16

7. VoIP e Multimedia 19

- 7.1 sip — Enumerazione SIP 19
- 7.2 rtsp — Streaming RTSP 21
- 7.3 onvif — Telecamere ONVIF 22
- 7.4 voip-enum ✦ NUOVO 23

8. Brute Force e Credenziali 25

- 8.1 creds, ssh, ftp, telnet, smb, rdp, vnc, ldap 25
- 8.2 Credential Store e Propagazione ✦ NUOVO 29

9. Protocolli IoT e Industriali 31

9.1 snmp, mqtt, modbus, coap, upnp, tftp, ipmi, bacnet, enip, redis 31

10. Telecamere e Device CVE 37

10.1 cam-cve — CVE Scanner 37

10.2 firmware ✦ NUOVO 39

10.3 config-audit ✦ NUOVO 40

11. Post-Exploitation e Firewall 41

12. Script Ausiliari Python / JS / Shell ✦ NUOVO 44

13. Pipeline Automatizzata 50

14. Orchestrazione: auto, resume, schedule, history ✦ NUOVO 52

15. Report CVSS 3.1 — Formato Italiano 55

15.1 diff — Confronto Assessment ✦ NUOVO 57

16. Scenari Pratici di Laboratorio 58

17. Cheat Sheet Rapido 63

18. Changelog 66

1. Introduzione e Concetti Base

1.1 Cos'è exa-dune?

exa-dune è un framework di penetration testing scritto in Bash (~11.700 righe), progettato per ambienti **IoT** e **VoIP**. Integra e coordina oltre 40 tool di sicurezza di Kali Linux attraverso un'interfaccia unificata, aggiunge 14 script ausiliari custom (Python, JavaScript, Shell), gestisce il ciclo completo di assessment e genera report PDF professionali in italiano con scoring CVSS 3.1.

> **NOTA**
>
> Questo strumento è destinato **esclusivamente** a sistemi di tua proprietà o per i quali hai esplicita autorizzazione scritta. L'uso non autorizzato è illegale.

1.2 Cosa può fare exa-dune v3.0?

- **51 subcomandi** — dalla scansione alla post-exploitation
- **14 script ausiliari** — custom Python/JS/Shell per attacchi specializzati
- **Profili auto** — voip-gateway, ip-camera, iot-adapter (lancia automaticamente i moduli giusti)
- **Propagazione credenziali** — credenziali trovate vengono testate su tutti i servizi del target
- **Pipeline automatizzata** — scan → attacco → post-exploit → PDF report con un solo comando
- **CVE database integrato** — Hikvision, Dahua, Axis, Reolink, AudioCodes MP20x, Commend ET962
- **Analisi firmware** — binwalk, credenziali hardcoded, binari SUID, certificati TLS
- **Storico SQLite** — tutti gli assessment indicizzati e confrontabili nel tempo
- **Scheduling cron** — assessment periodici automatici con diff automatico

1.3 Device supportati dal lab

DEVICE	MODELLO	PROTOCOLLI PRINCIPALI	PROFILO CONSIGLIATO
VoIP Gateway	AudioCodes MP20x	SIP, SNMP, HTTP, TFTP	`voip-gateway`
SIP Station	Commend ET962-BF5xx	SIP, HTTP, RTSP	`voip-gateway`
Commander VoIP	GAI-Tronics	SIP, HTTP	`voip-gateway`
IP Camera	Axis (vari)	RTSP, ONVIF, HTTP, SNMP	`ip-camera`
IP Camera	Reolink (vari)	RTSP, ONVIF, HTTP	`ip-camera`
IoT Adapter	Sconosciuto	variabile	`iot-adapter`

2. Installazione e Configurazione

2.1 Installazione dal repo GitHub

```
# Clone completo con script ausiliari
git clone https://github.com/Sal1699/exa-dune.git
cd exa-dune
sudo cp exa-dune /usr/local/bin/exa-dune
chmod +x /usr/local/bin/exa-dune

# Installa tool mancanti + script ausiliari in /usr/share/exa-dune/scripts/
sudo exa-dune install

# Verifica
exa-dune --version
exa-dune self-test
```

2.2 Aggiornamento automatico

```
# Aggiorna da GitHub (git pull + copia in /usr/local/bin + aggiorna script)
exa-dune update
```

2.3 Dipendenze Python opzionali

```
pip3 install paho-mqtt scapy
```

2.4 Dipendenze Node.js opzionali

```
apt install nodejs
npm install -g ws
```

2.5 Struttura directory

```
/usr/local/bin/exa-dune          # script principale
/usr/share/exa-dune/
  scripts/
    python/                       # 10 script Python
    js/                           # 2 script Node.js
    sh/                           # 2 script shell
  wordlists/                      # wordlist scaricate da SecLists

/root/pentest/exa-dune-<target>_<ts>/    # output assessment
  scan/                           # output modulo scan
  sip/                            # output modulo sip
  ...
  exa-dune.log                    # log completo della sessione
  findings.txt                    # tutti i finding per severità
  creds.json                      # credenziali trovate
  session.json                    # stato sessione (per resume)

/home/kali/Desktop/EXA-DUNE-Reports/     # report PDF generati
/root/pentest/exa-dune.db               # database SQLite storico
```

2.6 Self-test

```
exa-dune self-test                # verifica disponibilità tool
exa-dune self-test --install-missing  # installa automaticamente i mancanti
```

Verifica 33 tool tra cui: nmap, hydra, nikto, gobuster, svmap/svwar/svcrack, mosquitto, ffmpeg, testssl.sh, sslscan, binwalk, sqlite3, python3, node e molti altri.

3. Architettura e Struttura

3.1 Componenti principali

COMPONENTE	DESCRIZIONE
`exa-dune`	Script Bash principale (~11.700 righe, 51 subcomandi)
`scripts/python/`	10 script Python per attacchi specializzati
`scripts/js/`	2 script Node.js (WebSocket, XSS)
`scripts/sh/`	2 script shell helper (VoIP capture, net recon)
`creds.json`	Credential store per sessione (propagazione automatica)
`session.json`	Stato sessione corrente (per resume)
`exa-dune.db`	Database SQLite storico di tutti gli assessment
Wordlist embedded	8 categorie di wordlist integrate nello script (funziona senza SecLists)

3.2 Flusso di esecuzione tipico

scan → auto-suggest → attacchi mirati → cred propagation → post-exploit → PDF report

3.3 Credential Store (novità v3.0)

Ogni credenziale trovata durante il test viene salvata in `creds.json`. Al termine (o con `--propagate`), exa-dune testa automaticamente ogni coppia user:pass su tutti i servizi aperti del target:

```
[OK] snmp — community "public" trovata → salvata in creds.json
[OK] ssh  — admin:admin123 trovata (hydra) → salvata in creds.json

── Propagazione credenziali su 192.168.1.10 ──
[FOUND:CRITICAL] Credential reuse: admin:admin123 funziona su SSH (192.168.1.10
[FOUND:HIGH]     Credential reuse: admin:admin123 funziona su HTTP (192.168.1.10
[FOUND:INFO]     Telnet: nessuna corrispondenza
```

4. Flag Globali e Opzioni

4.1 Flag disponibili su tutti i moduli

FLAG	DESCRIZIONE
`--dry-run`	Mostra i comandi che verrebbero eseguiti senza eseguirli
`--stealth`	Modalità silenziosa: nmap -T2, rate nmap ridotto a 300/500 pps
`--adaptive`	NUOVO Riduce automaticamente il rate nmap quando rileva TCP reset (>10%)
`--timeout N`	Timeout in minuti per ogni tool (default: nessuno)
`--output DIR`	Directory di output (default: /root/pentest/exa-dune-<target>_<ts>/)
`--json`	NUOVO Scrive findings.json e summary.json per ogni modulo
`--help / -h`	Mostra l'help del subcomando specifico

4.2 Esempio con flag combinati

```
# Scan silenzioso con adaptive rate, timeout 30 min, output JSON
exa-dune scan 192.168.1.10 --stealth --adaptive --timeout 30 --json

# Dry-run per vedere cosa farebbe il modulo web
exa-dune web 192.168.1.10 --port 443 --scheme https --dry-run
```

4.3 Rate limiting adattivo (--adaptive)

Quando `--adaptive` è attivo, dopo ogni scan nmap exa-dune analizza il log cercando TCP reset e host irresponsivi:

- Se >10% dei pacchetti riceve TCP reset → riduce `NMAP_RATE_FAST` del 20%
- Se l'host risulta irresponsivo → passa automaticamente a `-T2` (stealth)
- Il rate minimo è 50 pps (fast) e 100 pps (full) per evitare blocchi completi

5. Profili Auto

I profili permettono di lanciare automaticamente la sequenza di moduli ottimale per tipo di device.

5.1 voip-gateway

Per: AudioCodes MP20x, Commend SIP Station ET962, GAI-Tronics Commander VoIP

```
exa-dune auto 192.168.1.10 --profile voip-gateway
```

Sequenza: scan → sip → snmp → ssh → telnet → web (80/443/8080) → tftp → ssl → creds → cam-cve (--vendor audiocodes) → voip-enum → propagazione credenziali → report PDF

5.2 ip-camera

Per: Axis (vari), Reolink (vari)

```
exa-dune auto 192.168.1.20 --profile ip-camera
```

Sequenza: scan → rtsp → onvif → cam-cve → web → snmp → ssl → creds → firmware (se disponibile) → propagazione credenziali → report PDF

5.3 iot-adapter

Per: device IoT generici con protocolli variabili

```
exa-dune auto 192.168.1.30 --profile iot-adapter
```

Sequenza: scan → sip → snmp → mqtt → http → tftp → ssh → creds → propagazione credenziali → report PDF

5.4 Profili esistenti

PROFILO	CASO D'USO
`voip-gateway` ✦	Gateway SIP, centralini VoIP
`ip-camera` ✦	Telecamere IP, NVR
`iot-adapter` ✦	Adapter IoT generici
`router`	Router di rete, access point
`generic`	Device sconosciuto, scansione completa

✦ NUOVO aggiunto in questa versione

5.5 Network sweep con auto-profiling

```
# Scansiona l'intera subnet, fingerprinta ogni host e lancia il profilo giusto
exa-dune network-sweep 192.168.1.0/24 --auto

# Solo tabella (senza lancia auto)
exa-dune network-sweep 192.168.1.0/24
```

Output esempio:

```
IP              MAC               TIPO           PORTE
192.168.1.10    00:90:8f:xx:xx    voip-gateway   22,80,5060
192.168.1.20    ac:cc:8e:xx:xx    ip-camera      80,554,8000
192.168.1.34    ec:71:db:xx:xx    ip-camera      80,554,8000,8080
```

6. Ricognizione e Scansione

scan

Port scan + fingerprinting + recon (nmap, arp-scan, searchsploit)

```
exa-dune scan <TARGET> [--stealth] [--adaptive] [--udp-top N] [--no-full] [--
```

Esegue in sequenza: ARP scan, nmap fast (top 1000 + script), nmap full (-p-), nmap UDP top N, searchsploit da XML nmap. Al termine mostra una tabella di attacchi consigliati basata sulle porte trovate.

FLAG	DEFAULT	DESCRIZIONE
`--stealth`	off	nmap -T2, rate ridotto
`--adaptive`	off	Riduce rate automaticamente
`--udp-top N`	20	Numero porte UDP da scansionare
`--no-full`	off	Salta nmap -p- (più veloce)
`--vuln`	off	Esegui nmap --script vuln + searchsploit per banner

network-sweep

Sweep subnet: arp-scan + fingerprint + tabella host

NUOVO

```
exa-dune network-sweep <CIDR> [--iface eth0] [--auto] [--timeout N]
```

Individua tutti gli host attivi, ne identifica il tipo (VoIP, camera, IoT, router) basandosi sulle porte aperte, e produce una tabella con il comando suggerito per ciascun host. Con `--auto` lancia automaticamente il profilo appropriato.

6.3 web, ssl, dns, js, http

```
# Audit web completo su porta 443
exa-dune web 192.168.1.10 --port 443 --scheme https --fuzz

# Analisi TLS completa
exa-dune ssl 192.168.1.10 --port 443

# DNS enumeration + zone transfer
exa-dune dns 192.168.1.10

# JavaScript analysis (endpoint, token, secret)
exa-dune js 192.168.1.10 --url http://192.168.1.10/app.js

# HTTP exploitation embedded (CGI, injection, auth bypass)
exa-dune http 192.168.1.10 --port 80
```

7. VoIP e Multimedia

sip

SIP/VoIP enumeration (svmap, svwar, svcrack, nmap SIP NSE)

```
exa-dune sip <TARGET> [--port 5060] [--range 100-500] [--transport udp|tcp]
```

- svmap: fingerprint del server SIP (vendor, versione)
- svwar: enumerazione extension nel range specificato
- svcrack: brute force password per le extension trovate
- Probe SIP manuale: REGISTER, OPTIONS, INVITE, INFO, SUBSCRIBE
- Cattura digest challenge per crack offline
- Toll fraud check: tenta chiamata senza autenticazione
- **sip_enum.py**: user enumeration avanzata con timing analysis (se installato)

rtsp

RTSP stream audit + Digest Auth brute force

```
exa-dune rtsp <TARGET> [--port 554] [--wordlist FILE]
```

- Enumera path RTSP (wordlist + path vendor-specific embedded)
- Digest Auth brute force via Python (MD5 socket raw)
- Snapshot capture con ffmpeg (proof-of-concept)

- **rtsp_fingerprint.py**: identifica vendor da RTSP OPTIONS/DESCRIBE (17 vendor)

`onvif` ONVIF/gSOAP IP camera audit (WS-Security brute force)

```
exa-dune onvif <TARGET> [--port 80] [--users FILE] [--pass FILE]
```

- Probe anonimo: GetDeviceInformation, GetCapabilities, GetProfiles
- Brute force WS-Security UsernameToken con digest SHA1
- **onvif_brute.py**: testa prima 14 coppie default hardcoded, poi wordlist

`voip-enum` VoIP enumeration avanzata NUOVO

```
exa-dune voip-enum <TARGET> [--port 5060] [--range 100-500] [--flood-test]
```

- **SIP OPTIONS flood**: 100 richieste in 10s, verifica rate limiting (429)
- **User enumeration**: distingue 401/403/404 per extension → identifica utenti validi
- **Toll fraud**: SIP INVITE senza auth → `[FOUND:CRITICAL]` se accettato
- **Extension scan**: svwar su range configurabile
- **TLS SIP**: verifica porta 5061, analisi certificato

> ATTENZIONE

Il flood test può saturare dispositivi VoIP con hardware limitato. Usa `--flood-test` solo se necessario.

8. Brute Force e Credenziali

8.1 Moduli di autenticazione

COMANDO	PROTOCOLLO	PORTA DEFAULT	NOTE
`creds`	multiprotocollo (hydra)	variabile	ssh, ftp, telnet, http, smb, rdp, vnc, sip, snmp, rtsp
`ssh`	SSH	22	ssh-audit per algoritmi + hydra + timing user enum
`ftp`	FTP	21	test anonimo, directory traversal, hydra
`telnet`	Telnet	23	timing attack, null auth, banner fingerprint, IoT stuffing
`smb`	SMB/CIFS	445	enum4linux, GPP password (MS14-025), signing check, named pipes
`rdp`	RDP	3389	NLA/CredSSP check, BlueKeep, RC4 detection
`vnc`	VNC	5900	nmap NSE, hydra
`ldap`	LDAP	389	anonymous bind, user/group enum

```
# Brute force SSH con wordlist custom
exa-dune creds 192.168.1.10 ssh --userlist /path/users.txt --passlist /path/pas
```

```
# Brute force multiprotocollo con propagazione
exa-dune creds 192.168.1.10 http --port 80 --propagate

# SMB con check GPP e signing
exa-dune smb 192.168.1.10

# Telnet con timing attack
exa-dune telnet 192.168.1.10 --timing-attack
```

8.2 Credential Store e Propagazione

Il credential store è un file JSON (`creds.json`) mantenuto nella directory di sessione. Ogni modulo che trova credenziali le salva automaticamente:

MODULO	COSA SALVA	CHIAVE SOURCE
snmp	community string valida	snmp
ssh	user:pass da hydra	ssh:hydra
ftp	accesso anonimo o user:pass	ftp:anon / ftp:hydra
telnet	user:pass da IoT stuffing o hydra	telnet:iot-stuffing / telnet:hydra
smb	user:pass da hydra	smb:hydra
rdp	user:pass da hydra	rdp:hydra
creds	qualsiasi protocollo	creds:hydra
sip	extension:password	sip:svcrack

Con `--propagate` (o automaticamente nei profili voip-gateway e ip-camera), le credenziali vengono testate su SSH, FTP, HTTP (80/443/8080/8443), Telnet:

```
exa-dune creds 192.168.1.10 ssh --propagate
```

Il sommario credenziali viene sempre incluso nel report PDF finale.

9. Protocolli IoT e Industriali

9.1 Panoramica moduli IoT/industriali

COMANDO	PROTOCOLLO	PORTA	HIGHLIGHTS
`snmp`	SNMP v1/v2c/v3	161/UDP	community brute, snmpwalk, SNMPv3, write test (snmpset)
`mqtt`	MQTT	1883/8883	accesso anonimo, subscribe #, retained dump, payload injection; **mqtt_audit.py**
`modbus`	Modbus TCP	502	FC1-FC43, Unit ID enum; **modbus_rw.py** per FC5/FC6 write PoC
`coap`	CoAP/UDP	5683	discovery, DTLS bypass, OBSERVE attack, PUT injection; **coap_scan.py**
`upnp`	UPnP/SSDP	1900/UDP	discovery, IGD, WANIPConnection exploit; **upnp_exploit.py**
`tftp`	TFTP/UDP	69	enum file comuni (config, firmware), download
`ipmi`	IPMI/BMC	623/UDP	cipher 0 bypass, default creds, hash dump
`bacnet`	BACnet/IP	47808/UDP	Who-Is broadcast, device object read
`enip`	EtherNet/IP	44818	ListIdentity, CIP object enum (PLC Allen-Bradley)

`redis`	Redis	6379	no-auth check, config dump, RCE via CONFIG SET dir/dbfilename
`wsd`	WS-Discovery	3702/UDP	multicast probe, device description
`mdns`	mDNS/ Bonjour	5353/UDP	service discovery locale

```
# SNMP community brute + walk OID
exa-dune snmp 192.168.1.10

# MQTT: accesso anonimo + dump retained messages
exa-dune mqtt 192.168.1.10

# Modbus: enum + write PoC
exa-dune modbus 192.168.1.10
python3 /usr/share/exa-dune/scripts/python/modbus_rw.py 192.168.1.10 --action w

# CoAP: discovery completa senza libcoap
python3 /usr/share/exa-dune/scripts/python/coap_scan.py 192.168.1.10 --action d

# UPnP: prova AddPortMapping senza auth
python3 /usr/share/exa-dune/scripts/python/upnp_exploit.py 192.168.1.10 --actio
```

10. Telecamere e Device CVE

cam-cve

CVE scanner per IP camera e device VoIP

```
exa-dune cam-cve <TARGET> [--vendor auto|hikvision|dahua|axis|reolink|audiocc
```

VENDOR	CVE	TIPO	CVSS
Hikvision	CVE-2021-36260	RCE via /SDK/webLanguage (XML injection)	9.8 CRITICAL
Hikvision	CVE-2017-7921	Auth bypass snapshot senza credenziali	9.8 CRITICAL
Dahua	CVE-2021-33044	Auth bypass magic packet	9.8 CRITICAL
Dahua	CVE-2017-7925	Config download senza auth	7.5 HIGH
Axis	CVE-2018-10660	VAPIX root shell via pwdgrp.cgi	9.8 CRITICAL
Axis	CVE-2016-6201	Snapshot auth bypass	7.5 HIGH
Reolink	—	RTSP senza auth, creds in URL	7.5 HIGH
AudioCodes MP20x	CVE-2019-9955	XSS in /Forms/login	6.1 MEDIUM

AudioCodes MP20x	CVE-2016-4960	Directory traversal firmware upgrade	7.5 HIGH
Commend ET962	—	Default creds, CGI paths, RTSP no-auth	9.8 CRITICAL

Il rilevamento vendor è automatico: analizza HTTP headers, SNMP sysDescr, RTSP OPTIONS.

```
# Auto-detect vendor + tutti i CVE applicabili
exa-dune cam-cve 192.168.1.20

# Batch su tutta la subnet
python3 /usr/share/exa-dune/scripts/python/cam_cve_batch.py 192.168.1.0/24 --
```

firmware Analisi firmware IoT NUOVO

```
exa-dune firmware <FILE_O_URL> [--vendor audiocodes|reolink|axis|commend]
```

- Download automatico da URL con curl
- **binwalk -e**: estrazione filesystem + analisi entropia
- Ricerca credenziali hardcoded: `/etc/passwd`, `/etc/shadow`, pattern `password=`
- Chiavi private RSA/EC: `-----BEGIN PRIVATE KEY-----`
- Certificati TLS: analisi con openssl (self-signed, scaduti)
- Binari SUID inusuali nel filesystem estratto
- Regole vendor-specifiche per AudioCodes, Reolink, Axis, Commend

`config-audit` Audit file di configurazione NUOVO

```
exa-dune config-audit <DIR_O_FILE> [--recursive]
```

Analizza file di configurazione scaricati (via TFTP/FTP/web) cercando:

- Credenziali in chiaro (pattern `password=`, `passwd:`, `secret=`) → CRITICAL
- Community SNMP deboli (`public`, `private`) → HIGH
- SIP con UDP senza TLS → MEDIUM
- Classificazione hash: `$1$`=MD5(MEDIUM), `$2a$`=bcrypt(INFO), DES(HIGH), plaintext(CRITICAL)
- Chiavi private esposte, token JWT, AWS keys → CRITICAL
- Telnet abilitato, SSH PasswordAuthentication yes → HIGH/MEDIUM
- Parser strutturati: JSON, XML, YAML, INI/CFG

11. Post-Exploitation e Firewall

11.1 post

```
exa-dune post <TARGET> <METODO> [--user U] [--pass P] [--privesc] [--exfil] [--
```

METODO	DESCRIZIONE
`shell`	Shell SSH interattiva + upload tool
`web`	Web shell upload su device con FTP/HTTP
`ftp`	FTP file exfiltration + config download
`smb`	SMB share enumeration + file download
`snmp`	SNMP MIB walk + set config
`revshell`	Genera payload reverse shell (Python/bash/PHP)
`privesc`	NUOVO Linux privesc: SUID, sudo -l, cron, world-writable

```
# Linux privilege escalation check
exa-dune post 192.168.1.10 shell --user admin --pass admin123 --privesc

# Windows pass-the-hash via impacket
exa-dune post 192.168.1.10 smb --pass-the-hash aad3b435b51404eeaad3b435b51404ee

# Cerca file sensibili (*.key, *.pem, *.db, *.cfg)
exa-dune post 192.168.1.10 shell --user admin --pass admin123 --exfil
```

11.2 fw — Firewall bypass

```
exa-dune fw <TARGET> [--stealth]
```

Tecniche di evasione firewall testate:

- ACK/FIN/NULL/XMAS scan per rilevare regole stateful vs stateless
- IP fragmentation bypass
- Source port spoofing (porta 53, 80, 443)
- IPv6 bypass (se dual-stack)
- UPnP pinhole (aggira NAT via WANIPConnection)
- SIP ALG detection e bypass
- TTL-based route analysis

12. Script Ausiliari Python / JS / Shell

Installati in `/usr/share/exa-dune/scripts/` tramite `exa-dune install`. Vengono richiamati automaticamente dai moduli principali quando disponibili, e possono essere usati in modo standalone.

12.1 Script Python

sip_enum.py — SIP User Enumeration

```
python3 sip_enum.py <TARGET> [--port 5060] [--range 100-500] [--method register
```

Enumera extension SIP via REGISTER/OPTIONS. Distingue 401 (utente valido), 403 (bloccato), 404 (non trovato). Include timing analysis: se la differenza di risposta tra utenti è >2 deviazioni standard segnala user enumeration via timing.

onvif_brute.py — ONVIF Default + Brute Force

```
python3 onvif_brute.py <TARGET> [--port 80] [--users FILE] [--passwords FILE]
```

Testa prima 14 coppie default hardcoded (admin:admin, admin:12345, Admin:admin...). Poi brute force con wordlist. Genera envelope SOAP WS-Security con UsernameToken + Nonce + SHA1 PasswordDigest. Se trova credenziali valide: esegue GetDeviceInformation e GetCapabilities.

modbus_rw.py — Modbus Read/Write PoC

```
python3 modbus_rw.py <TARGET> [--port 502] [--unit 1] [--action read|write|enum
```

Implementa FC1-FC4 (read), FC5 (Write Single Coil), FC6 (Write Single Register), FC43 (Device Identification) via socket raw o pymodbus se

installato. Il write scrive valore 0x0000 (safe per PoC). Segnala CRITICAL se il write riesce senza autenticazione.

coap_scan.py — CoAP Client Custom

```
python3 coap_scan.py <TARGET> [--port 5683] [--action discover|get|put|observe]
```

Implementa il protocollo CoAP manualmente (senza libcoap). Supporta: discovery via `/.well-known/core`, GET di risorse, PUT injection, OBSERVE per notifiche continue. Rileva risorse sensibili (`/auth`, `/admin`, `/config`).

rtsp_fingerprint.py — Vendor Fingerprint

```
python3 rtsp_fingerprint.py <TARGET> [--port 554]
```

Invia RTSP OPTIONS e DESCRIBE, analizza header `Server:` e `Via:`. Riconosce 17 vendor: Hikvision, Dahua, Axis, Reolink, AudioCodes, Commend, Bosch, Hanwha, Milestone, VxWorks e altri. Dopo l'identificazione testa i path RTSP specifici del vendor.

mqtt_audit.py — MQTT Deep Audit

```
python3 mqtt_audit.py <TARGET> [--port 1883] [--tls] [--user U] [--pass P]
```

Usa paho-mqtt. Testa accesso anonimo, subscribe su `#` e `$SYS/#`, analizza i messaggi ricevuti cercando pattern sensibili (password, token, key). Testa write su topic `cmd/`, `set/`, `control/`, `actuator/`.

upnp_exploit.py — UPnP IGD Exploit

```
python3 upnp_exploit.py <TARGET> [--action discover|addport|getip|enum]
```

Discovery SSDP, GetExternalIPAddress (rivela IP WAN), AddPortMapping senza autenticazione. Cleanup automatico con DeletePortMapping dopo il test.

cam_cve_batch.py — CVE Batch Scanner

```
python3 cam_cve_batch.py <IP_LIST_O_CIDR> [--vendor auto] [--output results.jso
```

Accetta CIDR o file con lista IP. Per ogni host: fingerprinta il vendor, testa tutti i CVE applicabili (Hikvision, Dahua, Axis, Reolink, AudioCodes, Commend). Output tabella + JSON opzionale.

http_iot_fuzz.py — HTTP Fuzzer Embedded

```
python3 http_iot_fuzz.py <TARGET> [--port 80] [--scheme http|https] [--mode pat
```

60+ CGI path comuni per device embedded. Injection via GET con payload `| id`, `;id`, `$(id)`. Auth bypass via header manipulation (X-Forwarded-For, X-Real-IP). Lista default creds IoT per HTTP Basic Auth.

sip_pcap_creds.py — SIP PCAP Credential Extractor

```
python3 sip_pcap_creds.py <PCAP_FILE> [--wordlist FILE] [--output creds.txt]
```

Usa scapy per leggere PCAP. Estrae header `Authorization: Digest` dai pacchetti SIP. Tenta crack offline del hash MD5 (username:realm:password format). Output in formato hashcat 11400 per crack avanzato.

12.2 Script JavaScript (Node.js)

websocket_audit.js — WebSocket Auditor

```
node websocket_audit.js <TARGET> [--port 8080] [--path /ws] [--mode probe|fuzz|
```

Testa connessioni WebSocket su 14 path comuni (`/ws`, `/live`, `/stream`, `/api/ws`...). Modalità probe: cattura messaggi iniziali. Modalità auth: verifica se i dati vengono inviati senza token. Modalità fuzz: invia JSON malformati. Supporta ws:// e wss://.

xss_generator.js — XSS Payload Generator

```
node xss_generator.js [--target URL] [--param field] [--mode test|steal|exec]
```

Genera payload XSS in 5 encoding diversi (HTML entities, URL, unicode, hex, base64). Con `--target` testa direttamente la reflection via HTTP. Include test specifico per CVE-2019-9955 (AudioCodes XSS su `/Forms/login`).

12.3 Script Shell

voip_capture.sh — VoIP Traffic Capture

```
voip_capture.sh [--iface eth0] [--target 192.168.1.0/24] [--duration 60] [--outp
```

Cattura traffico SIP (5060/5061 TCP+UDP) e RTP (10000-20000 UDP) con tcpdump. Post-capture: analisi con tshark/sngrep, estrazione stream RTP se disponibile rtpdump, estrazione automatica credenziali SIP via sip_pcap_creds.py.

net_recon.sh — Quick Network Recon

```
net_recon.sh <CIDR> [--output DIR]
```

arp-scan per host discovery, nmap ping sweep parallelo, nmap top-50 ports per ogni host vivo, fingerprinting automatico tipo device. Output compatibile con `exa-dune network-sweep`.

13. Pipeline Automatizzata

```
exa-dune pipeline <TARGET> [--profile PROFILO] [--from-scan DIR] [--dry-run]
```

La pipeline esegue automaticamente tutte le fasi:

FASE 1

Scansione

nmap fast + full + UDP. Produce nmap_fast.nmap con porte aperte.

FASE 2

Auto-suggest

Analizza porte aperte e mappa a moduli exa-dune con priorità. Mostra tabella attacchi consigliati.

FASE 3

Attacchi in ordine di priorità

Esegue i moduli in sequenza: prima i più critici (telnet, SIP, SMB), poi gli altri.

FASE 4

Propagazione credenziali

Tutte le credenziali trovate vengono testate cross-service automaticamente.

FASE 5

Report HTML + PDF

Genera report professionale con executive summary, CVSS 3.1, findings table, credenziali trovate. PDF salvato su Desktop.

```
# Pipeline completa con profilo voip-gateway
exa-dune pipeline 192.168.1.10 --profile voip-gateway

# Riusa scan esistente (salta fase 1)
exa-dune pipeline 192.168.1.10 --from-scan /root/pentest/exa-dune-192-168-1-10_

# Dry-run per vedere cosa farebbe
exa-dune pipeline 192.168.1.10 --dry-run
```

14. Orchestrazione: resume, schedule, history, update

`resume` Riprende assessment interrotto NUOVO

```
exa-dune resume <DIR_SESSIONE>
```

Legge `session.json` nella directory, mostra i moduli già completati e riprende dall'ultimo punto. Ogni modulo completato aggiorna automaticamente session.json.

```
# Assessment interrotto dopo scan e sip
exa-dune resume /root/pentest/exa-dune-192-168-1-10_20260322/
# Output: "Moduli completati: scan, sip (2/8). Riprendo da: snmp..."
```

`schedule` Scan periodici via cron NUOVO

```
exa-dune schedule --add --target <IP> --cron "EXPR" --profile <PROFILO>
exa-dune schedule --list
exa-dune schedule --remove <ID>
```

Aggiunge cron job che esegue automaticamente la pipeline sul target. Dopo ogni run schedulata, esegue `exa-dune diff` contro la run precedente e salva il report differenziale.

```
# Assessment settimanale ogni lunedì alle 2:00
exa-dune schedule --add --target 192.168.1.10 --cron "0 2 * * 1" --profile vo

# Lista job attivi
exa-dune schedule --list
```

history Storico assessment SQLite NUOVO

```
exa-dune history [--target IP] [--last N] [--findings ID]
```

Tutti gli assessment vengono salvati automaticamente in `/root/pentest/exa-dune.db` (SQLite). Tabella `assessments`: target, modulo, timestamp, directory, conteggi finding. Tabella `findings`: ogni singolo finding con severità e messaggio.

```
# Ultimi 10 assessment
exa-dune history --last 10

# Tutti gli assessment sul target
exa-dune history --target 192.168.1.10

# Finding di un assessment specifico
exa-dune history --findings 42
```

update Aggiornamento automatico NUOVO

```
exa-dune update
```

Trova il repo git locale dello script, esegue `git pull origin main`, copia l'aggiornamento in `/usr/local/bin/exa-dune`, aggiorna anche gli script ausiliari. Se il repo non è trovato, scarica direttamente da GitHub raw.

15. Report CVSS 3.1 — Formato Italiano

```
exa-dune report <DIR_SESSIONE> [--pdf] [--open]
```

15.1 Struttura del report

1. **Copertina** — Target, data, classificazione CONFIDENZIALE
2. **Sommario Esecutivo** — Testo narrativo auto-generato: "Il target X espone Y vulnerabilità critiche..."
3. **Conteggio per severità** — CRITICAL / HIGH / MEDIUM / LOW / INFO con contatori
4. **Tabella Finding** — ID, Severità, CVSS Score, Titolo, Descrizione
5. **Dettaglio CRITICAL/HIGH** — Descrizione estesa + raccomandazione per ogni finding grave
6. **Credenziali Trovate** — Tabella con service, host, porta, user, password, source
7. **Appendice** — Output raw dei tool (collassabile)

15.2 Scoring CVSS 3.1

SEVERITÀ FINDING	CVSS BASE SCORE	COLORE
CRITICAL	9.8	CRITICAL
HIGH	7.5	HIGH
MEDIUM	5.5	MEDIUM
LOW	2.5	LOW

INFO	0.0	INFO

15.3 Conversione HTML → PDF

Priorità tool: weasyprint → wkhtmltopdf → chromium. Il PDF viene salvato automaticamente in `/home/kali/Desktop/EXA-DUNE-Reports/REPORT_<target>_<data>.pdf`.

```
# Genera report HTML + PDF e apre il PDF
exa-dune report /root/pentest/exa-dune-192-168-1-10_20260322/ --pdf
```

diff Confronto differenziale tra due assessment NUOVO

```
exa-dune diff <DIR-OLD> <DIR-NEW> [--pdf]
```

Confronta i `findings.txt` di due sessioni diverse. Calcola finding nuovi (PEGGIORA), finding risolti (MIGLIORA), e finding invariati (STABILE). Confronta anche le porte aperte. Con `--pdf` genera report differenziale.

```
exa-dune diff /root/pentest/run_gennaio/ /root/pentest/run_marzo/ --pdf
```

16. Scenari Pratici di Laboratorio

Scenario 1 — Audit Completo AudioCodes MP20x

192.168.1.10

```
# Opzione 1: pipeline automatica
exa-dune pipeline 192.168.1.10 --profile voip-gateway

# Opzione 2: manuale step-by-step
exa-dune scan 192.168.1.10 --vuln
exa-dune sip 192.168.1.10 --range 100-299
exa-dune snmp 192.168.1.10
exa-dune web 192.168.1.10 --port 80 --scheme http
exa-dune cam-cve 192.168.1.10 --vendor audiocodes
exa-dune voip-enum 192.168.1.10 --flood-test
exa-dune config-audit /root/pentest/exa-dune-192-168-1-10*/tftp/ --recursive
exa-dune creds 192.168.1.10 ssh --propagate
exa-dune report /root/pentest/exa-dune-192-168-1-10*/ --pdf
```

Scenario 2 — Audit Telecamera Reolink

192.168.1.34

```
exa-dune auto 192.168.1.34 --profile ip-camera

# O manuale:
exa-dune scan 192.168.1.34
exa-dune rtsp 192.168.1.34 --port 554
python3 /usr/share/exa-dune/scripts/python/rtsp_fingerprint.py 192.168.1.34
exa-dune onvif 192.168.1.34 --port 8000
python3 /usr/share/exa-dune/scripts/python/onvif_brute.py 192.168.1.34
exa-dune cam-cve 192.168.1.34 --vendor reolink
```

```
exa-dune web 192.168.1.34 --port 80
exa-dune report /root/pentest/exa-dune-192-168-1-34*/ --pdf
```

Scenario 3 — Sweep subnet completo `192.168.1.0/24`

```
# Scopri tutti gli host e lancia il profilo appropriato automaticamente
exa-dune network-sweep 192.168.1.0/24 --auto

# O usa lo script shell per recon rapido
/usr/share/exa-dune/scripts/sh/net_recon.sh 192.168.1.0/24
```

Scenario 4 — Analisi firmware scaricato

```
# Scarica firmware da TFTP e analizza
exa-dune tftp 192.168.1.10              # scarica config/firmware
exa-dune firmware /root/pentest/exa-dune-192-168-1-10*/tftp/firmware.bin --ve

# Audit file di configurazione scaricati
exa-dune config-audit /root/pentest/exa-dune-192-168-1-10*/tftp/ --recursive
```

Scenario 5 — Cattura e analisi traffico VoIP

```
# Cattura traffico SIP+RTP per 60 secondi
/usr/share/exa-dune/scripts/sh/voip_capture.sh --iface eth0 --target 192.168.

# Estrai credenziali dal PCAP catturato
python3 /usr/share/exa-dune/scripts/python/sip_pcap_creds.py /root/voip_captu
  --wordlist /usr/share/wordlists/fasttrack.txt
```

Scenario 6 — Monitoring continuo con diff automatico

```
# Assessment settimanale + diff automatico
exa-dune schedule --add --target 192.168.1.10 --cron "0 2 * * 1" --profile vo

# Confronto manuale tra due run
exa-dune diff /root/pentest/run_gennaio/ /root/pentest/run_marzo/ --pdf

# Storico completo
exa-dune history --target 192.168.1.10 --last 10
```

17. Cheat Sheet Rapido

RICOGNIZIONE

```
exa-dune scan TARGET
exa-dune scan TARGET --vuln --adaptive
exa-dune network-sweep 192.168.1.0/24
exa-dune network-sweep CIDR --auto
exa-dune web TARGET --port 443 --scheme https
exa-dune ssl TARGET
exa-dune dns TARGET
```

VOIP / SIP

```
exa-dune sip TARGET --range 100-500
exa-dune rtsp TARGET
exa-dune onvif TARGET --port 8000
exa-dune voip-enum TARGET --flood-test
exa-dune sip-fuzz TARGET
python3 sip_enum.py TARGET --range 100-
python3 onvif_brute.py TARGET
```

BRUTE FORCE

```
exa-dune creds TARGET ssh
exa-dune creds TARGET http --port 80
exa-dune creds TARGET --propagate
exa-dune ssh TARGET
exa-dune smb TARGET
exa-dune telnet TARGET --timing-attack
exa-dune rdp TARGET --check-nla
```

IOT / INDUSTRIALE

```
exa-dune snmp TARGET
exa-dune mqtt TARGET
exa-dune modbus TARGET
exa-dune coap TARGET
exa-dune upnp TARGET
python3 modbus_rw.py TARGET --action wr
python3 mqtt_audit.py TARGET
python3 upnp_exploit.py TARGET --action
```

CAMERA CVE

```
exa-dune cam-cve TARGET
exa-dune cam-cve TARGET --vendor hikvision
exa-dune cam-cve TARGET --vendor audiocodes
exa-dune cam-cve TARGET --vendor commend
python3 cam_cve_batch.py 192.168.1.0/24
python3 rtsp_fingerprint.py TARGET
```

FIRMWARE / CONFIG

```
exa-dune firmware firmware.bin --vendor
exa-dune config-audit /path/to/configs/
python3 http_iot_fuzz.py TARGET --mode
python3 sip_pcap_creds.py capture.pcap
node xss_generator.js --target http://T
node websocket_audit.js TARGET --port 8
```

PROFILI E PIPELINE

```
exa-dune auto TARGET --profile voip-gateway
exa-dune auto TARGET --profile ip-camera
exa-dune auto TARGET --profile iot-adapter
exa-dune pipeline TARGET
exa-dune pipeline TARGET --from-scan DIR
exa-dune resume DIR_SESSIONE
```

REPORT E STORICO

```
exa-dune report DIR --pdf
exa-dune diff DIR_OLD DIR_NEW --pdf
exa-dune history --last 10
exa-dune history --target TARGET
exa-dune history --findings ID
exa-dune schedule --list
exa-dune self-test --install-missing
```

Porte IoT/VoIP di riferimento

21 FTP | 22 SSH | 23 Telnet | 53 DNS | 69 TFTP | 80 HTTP
161 SNMP | 389 LDAP | 443 HTTPS | 445 SMB | 502 Modbus
554 RTSP | 623 IPMI | 1883 MQTT | 1900 UPnP | 3389 RDP
5060 SIP | 5061 SIP/TLS | 5683 CoAP | 5900 VNC | 6379 Redis
8000 ONVIF | 8554 RTSP alt | 44818 EtherNet/IP | 47808 BACnet

18. Changelog

v3.0 — 22 Marzo 2026

- **14 script ausiliari**: 10 Python, 2 Node.js, 2 Shell in `scripts/`
- **Installazione automatica script**: `exa-dune install` copia in `/usr/share/exa-dune/scripts/`
- **Potenziamento moduli**: telnet (timing attack, null auth, banner fingerprint), upnp (WANIPConnection exploit), rdp (NLA/CredSSP, BlueKeep), coap (DTLS bypass, OBSERVE), config-audit (JSON/XML/YAML parser, hash class, CIS), post (privesc, PTH), smb (GPP, signing check)
- **Credential propagation**: credential store cross-service automatico
- **8 nuovi subcomandi**: update, resume, schedule, firmware, voip-enum, config-audit, history, diff
- **3 nuovi profili auto**: voip-gateway, ip-camera, iot-adapter
- **network-sweep**: sweep subnet con fingerprint automatico
- **self-test**: verifica 33 tool con --install-missing

v2.5.0 — Marzo 2026

- Fix debug generale: 12 categorie di bug (quoting, injection, race condition, variabili)
- Nuovi subcomandi IoT/OT: mdns, wsd, bacnet, rtmp, cam-cve, sip-fuzz, enip, vnc, rdp, ldap, redis, http, post, fw
- Report HTML/PDF con CVSS 3.1 e template professionale italiano
- CVE AudioCodes MP20x e Commend ET962 in cam-cve
- Pipeline smart: scan → attack → post → PDF
- Wordlist scaricabili da SecLists GitHub

v2.0 — Febbraio 2026

- Primo rilascio pubblico su GitHub

- 29 subcomandi, wordlist embedded, pipeline automatizzata

REPOSITORY

GitHub: `https://github.com/Sal1699/exa-dune`

Aggiornamento: `exa-dune update`

Issues: segnala bug o richieste di funzionalità su GitHub Issues

---

⚠ CONFIDENZIALE — USO ESCLUSIVAMENTE IN AMBIENTI AUTORIZZATI ⚠